Upon receiving the yielded value as the return value of resume, our main program enters the body of the loop and increments the count for the first word returned by the fiber. It then loops back up to the top of the while loop, which again calls words.resume while evaluating the condition. The resume call goes back into the block, continuing where it left off at the line after the Fiber.yield call.

When the fiber runs out of words in the file, the foreach block exits, and the code in the fiber terminates. Just as with a method call, the return value of the fiber will be the value of the last expression evaluated (in this case the nil). In this case, the nil is not strictly needed, as foreach will return nil when it terminates. The nil just makes it explicit. The next time resume is called, it returns this value nil. You'll get a FiberError if you attempt to call resume again after the fiber has terminated.

Fibers can be used to generate values from infinite sequences on demand. Here's a fiber that returns successive integers divisible by 2 and not divisible by 3:

```
twos = Fiber.new do
  num = 2
  loop do
    Fiber.yield(num) unless num % 3 == 0
    num += 2
  end
end
10.times { print twos.resume, " " }
```

produces:

```
2 4 8 10 14 16 20 22 26 28
```

However, you can more easily use lazy enumerators to gracefully create infinite lists. These are described Enumerators Used as Generators and Filters, on page 80.

Because fibers are just objects, you can pass them around, store them in variables, and so on. Fibers can be resumed only in the thread that created them.

Fibers can also use the transfer method to explicitly transfer control between specific fibers. The tricky part here is that the receiver of transfer is the thread to be resumed—in other words, the call is fiber_that_gets_control.transfer(args) and not, calling_fiber.transfer(fiber_that_gets_control). The return value of the transfer call is the same as yield—the last expression before the fiber pauses control again.

The yield/resume mechanism for switching control and the transfer method don't work well together. Specifically, if a fiber is started with resume then cedes control, it can only receive control using the same mechanism. If it uses yield, it can only be returned with resume. If it transfers out, it can only be transfered back. If a fiber is started with transfer, it can only return control using transfer, not yield. Using the wrong mechanism will result in an exception.

In Ruby 3.0, fibers can be *non-blocking*, meaning that when a fiber would otherwise block because of I/O or waiting on another process it automatically cedes control to a fiber scheduler which choses another fiber to wake up and controls resuming the original fiber when it has whatever it needs to proceed.

To create a non-blocking Fiber, you need to do two things:

- Call Fiber.set_scheduler to set a scheduler
- Create the fiber with Fiber.new(blocking: false)